COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	8/7/26	**DATE SUBMITTED:**	8/8/26
NAME:	Francis Elijah J. Formalejo	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5 **Queues**

1. PROCEDURE OUTPUT

What procedures did you perform?

MAIN:

```
#include <iostream>
#include "queueLL.h"

int main() {

    qNode<int>* front = nullptr;
    qNode<int>* back = nullptr;

    enqueue(1, &front, &back);
    std::cout << "front: " << front->data << " back: " <<
    enqueue(2, &front, &back);
    enqueue(3, &front, &back);
    std::cout << "front: " << front->data << " back: " <<

    display(front);
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " <<
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " <<
    dequeue(&front, &back);
    dequeue(&front, &back);

    return 0;
}
```

QUEUELL:

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

#include <iostream>

template <typename T>
class qNode {
public:
    T data;
    qNode<T> *next;
};

// creating the new node
template <typename T>
qNode<T>* new_node(T newData) {
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr) {
    qNode<T>* newNode = new_node(newData);

    // inserting into an empty queue
    if ((*frontPtr) == nullptr) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }
```

```
32
33      // point the back next to the newNode
34      (*backPtr)->next = newNode;
35      // the newNode will be the new backPtr
36      (*backPtr) = newNode;
37  }
38
39  template <typename T>
40  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
41      // check if queue is empty
42      if ((*frontPtr) == nullptr) {
43          std::cout << "The queue is empty. Nothing to dequeue.\n";
44          return;
45      }
46
47      qNode<T>* tempPtr = (*frontPtr);
48
49      // check if queue has only 1 node left
50      if ((*frontPtr) == (*backPtr)) {
51          (*frontPtr) = nullptr;
52          (*backPtr) = nullptr;
53          delete tempPtr;
54          return;
55      }
56
57      // update frontPtr
```

```
58      (*frontPtr) = (*frontPtr)->next;
59      tempPtr->next = nullptr;
60      delete tempPtr;
61  }
62
63  // display all items in the queue
64  template <typename T>
65  void display(qNode<T>* frontPtr) {
66      if (frontPtr == nullptr) {
67          std::cout << "The queue is empty.\n";
68          return;
69      }
70
71      qNode<T>* current = frontPtr;
72      while (current != nullptr) {
73          std::cout << current->data << " ";
74          current = current->next;
75      }
76      std::cout << std::endl;
77  }
78
79  #endif // QUEUELL_H
```

QUEUEARR:

```
1   #ifndef QUEUEARR_H
2   #define QUEUEARR_H
3
4   #include <iostream>
5
6   template <typename T>
7   class queueArr {
8   private:
9       T* q_array;
10      size_t q_capacity;
11      size_t q_size;
12      int q_front;
13      int q_back;
14
15  public:
16      // constructor
17      queueArr(size_t capacity = 10);
18
19      bool Empty();
20      size_t Size();
21      void Clear();
22
23      T Front();
24      T Back();
25
26      void Enqueue(T value);
27      void Dequeue();
28
29      // copy constructor
30      queueArr(const queueArr& other);
```

```
32      // copy assignment operator
33      queueArr& operator=(const queueArr& other);
34
35      // destructor
36      ~queueArr();
37  };
38
39  // constructor
40  template <typename T>
41  queueArr<T>::queueArr(size_t capacity) {
42      q_capacity = capacity;
43      q_size = 0;
44      q_front = 0;
45      q_back = -1;
46      q_array = new T[q_capacity];
47  }
48
49  template <typename T>
50  bool queueArr<T>::Empty() {
51      return q_size == 0;
52  }
53
54  template <typename T>
55  size_t queueArr<T>::Size() {
56      return q_size;
57  }
58
59  template <typename T>
```

```
60  void queueArr<T>::Clear() {
61      q_size = 0;
62      q_front = 0;
63      q_back = -1;
64  }
65
66  template <typename T>
67  T queueArr<T>::Front() {
68      if (Empty()) {
69          std::cout << "The Queue is Empty.\n";
70          return T{};
71      }
72      return q_array[q_front];
73  }
74
75  template <typename T>
76  T queueArr<T>::Back() {
77      if (Empty()) {
78          std::cout << "The Queue is Empty.\n";
79          return T{};
80      }
81      return q_array[q_back];
82  }
83
84  template <typename T>
85  void queueArr<T>::Enqueue(T value) {
86      if (q_size == q_capacity) {
87          std::cout << "Queue is full\n";
88          return;
```

```
89          }
90
91          q_back = (q_back + 1) % q_capacity;
92          q_array[q_back] = value;
93          q_size++;
94      }
95
96      template <typename T>
97      void queueArr<T>::Dequeue() {
98          if (Empty()) {
99              std::cout << "The queue is empty.\n";
100             return;
101         }
102
103         q_front = (q_front + 1) % q_capacity;
104         q_size--;
105
106         if (Empty()) {
107             Clear();
108         }
109     }
110
111     template <typename T>
112     queueArr<T>::~queueArr() {
113         delete[] q_array;
114     }
115
116     // Copy Constructor
117     template <typename T>
```

```cpp
queueArr<T>::queueArr(const queueArr& other) {
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    q_array = new T[q_capacity];
    for (size_t i = 0; i < q_capacity; ++i) {
        q_array[i] = other.q_array[i];
    }
}

// Copy Assignment Operator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other) {
    if (this != &other) {
        delete[] q_array;

        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;

        q_array = new T[q_capacity];
        for (size_t i = 0; i < q_capacity; ++i) {
            q_array[i] = other.q_array[i];
        }
    }
    return *this;
```

```cpp
    return *this;
}

#endif // QUEUEARR_H
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```cpp
#include <iostream>
#include <string>
#include "queueLL.h"

class Job {
public:
    int jobID;
    std::string userName;
    int pageCount;

    Job() : jobID(0), userName(""), pageCount(0) {}
    Job(int id, std::string name, int pages) : jobID(id), userName(name), pageCount(pages) {}
};

class Printer {
private:
    qNode<Job>* frontPtr;
    qNode<Job>* backPtr;

public:
    Printer() : frontPtr(nullptr), backPtr(nullptr) {}

    void addJob(int id, const std::string& name, int pages) {
        Job newJob(id, name, pages);
        enqueue(newJob, &frontPtr, &backPtr);
        std::cout << "Job ID " << id << " added by " << name << " (" << pages << " pages)\n";
    }

    void processJobs() {
        if (frontPtr == nullptr) {
            std::cout << "The queue is empty.\n";
```

Activat
Go to Set

```cpp
<< currentJob.userName << " (" << currentJob.pageCount << " pages)\n";
```

```
            return;
        }

        while (frontPtr != nullptr) {
            Job currentJob = frontPtr->data;
            std::cout << "Processing Job ID " << currentJob.jobID << " for " << currentJob.userName << " (" << currentJob
            dequeue(&frontPtr, &backPtr);
        }
    }
};

int main() {
    Printer officePrinter;

    officePrinter.addJob(101, "Alice", 5);
    officePrinter.addJob(102, "Bob", 12);
    officePrinter.addJob(103, "Charlie", 3);

    officePrinter.processJobs();

    return 0;
}
```

```
Job ID 101 added by Alice (5 pages)
Job ID 102 added by Bob (12 pages)
Job ID 103 added by Charlie (3 pages)
Processing Job ID 101 for Alice (5 pages)
Processing Job ID 102 for Bob (12 pages)
Processing Job ID 103 for Charlie (3 pages)
```

Analysis:
The program logs each print data from the computer like they falled in line. When the printer starts printing the data from the computer, it did Alice's first, then Bob and Charlie. This output happens because the program uses a queue data structure that works like the first come, first serve like program.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

COMPUTER ENGINEERING

QUEZON CITY

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion:

Overall, this lab activity was hard yet enjoyable for me. Enjoyable because the experience of how queue data structures works and applying them to a real life like the shared printer simulation was kinda cool. And also implementing the important things like enqueuing and dequeuing was my fav part. The hard part was making sure that the pointer management in the linked list was clean so that data was handled correctly. And hard because of the troubleshooting that I need to do before getting the output since I need to re-do my whole vscode all over again. Overall, this was a 7/10 for me because of the troubleshooting that I did, but the activity was fun tbh.